Design: Recording Finalization Reliability

Shipped design contract for sealing active and background recordings when the live KSP vessel ends before Parsek can run the normal scene-exit finalizer. The recording-finalization cache feature has landed and is treated as baseline behavior, with the known limitation below.

Related docs: `parsek-flight-recorder-design.md`, `parsek-rewind-to-separation-design.md`, `dev/done/plans/incomplete-ballistic-extrapolation.md`, `dev/done/plans/recording-finalization-reliability.md`, `dev/manual-testing/test-recording-finalization-cache.md`.

Status

Recording-finalization reliability is implemented and production behavior follows the precedence defined here:

1. Fresh live finalization wins when the vessel/runtime can still be inspected.
2. A matching finalization cache wins when the live finalizer declines or the vessel is missing.
3. Trajectory inference remains only a degraded last-resort fallback and must log that degradation.

The landed implementation includes:

- `RecordingFinalizationCache` for in-memory terminal payloads.
- `RecordingFinalizationCacheProducer` for active, loaded-background, and on-rails refreshes.
- `RecordingFinalizationCacheApplier` for identity checks, stale/invalid rejection, predicted-tail trimming, terminal metadata stamping, and dirty-sidecar handling.
- Active/background ownership transfer when a recording moves between focused and background sampling.
- Scene-exit, crash, background destroy, unload/missing-vessel, debris TTL, and out-of-bubble cache consumers.
- Maneuver-node-safe finalization: UI maneuver nodes are detected and stock node-contaminated tails are discarded in favor of current-state propagation.
- Headless xUnit coverage and `RecordingFinalization` in-game runtime canaries, with the remaining gameplay checklist kept in the manual-testing doc.

Known limitation:

- `docs/dev/done/todo-and-known-bugs-v5.md` item 639 tracked a patched-conic atmospheric-tail cache refresh that could flip from `Destroyed` to `Orbiting` from a far-future segment. It was closed 2026-05-10 as not-pursued: it is a diagnostic-only cache thrash, not a correctness bug, because the active-recorder `VesselDestroyedDuringRecording` override is authoritative. Reopen only if a future path comes to depend on that cache without the override.

Problem

Historically, Parsek could synthesize a predicted tail for some incomplete recordings when the player exited the flight scene while the live vessel still existed. That was not enough for mission-tree correctness. A recording can end because the player switches focus, a background sibling leaves the physics bubble, KSP deletes an unfocused atmospheric vessel, the active vessel crashes before scene exit, or `FlightGlobals` is unavailable during teardown. Without a cached terminal payload, those paths could commit only the last sampled point, infer a weak terminal state from stale data, or mark the vessel destroyed without the synthetic trajectory that explains how it got there.

This is foundational for Rewind to Separation. The Unfinished Flights classifier needs trustworthy terminal state and end-time data to decide whether a sibling really ended badly and whether a rewind point should stay actionable. A terminal state that depends on which KSP object happened to survive until scene exit is not a durable gameplay contract.

Landed Implementation Audit

The shipped implementation keeps the original scene-exit finalizer and extrapolator, and adds cache refresh/application around the timing and ownership gaps:

- `IncompleteBallisticSceneExitFinalizer.TryApply` snapshots a live vessel's patched-conic chain, then runs `BallisticExtrapolator.Extrapolate` when needed.
- `BallisticExtrapolator.ShouldExtrapolate` correctly targets flying, suborbital, escaping, and low-periapsis orbital cases while skipping landed, splashed, prelaunch, docked, and stable orbiting vessels.
- `BackgroundRecorder` records loaded physics-bubble siblings, on-rails orbit checkpoints, part death and joint-break events, debris TTL expiry, and background-vessel destruction.
- `FinalizeTreeRecordings` runs for scene exit, manual commit, revert commit, and post-destruction merge paths.
- `FlightRecorder` owns the focused recording cache and refreshes it at recording start, cadence, and lifecycle/state transitions.
- `BackgroundRecorder` owns per-PID caches for loaded and on-rails background recordings, including inherited caches from active → background transitions.
- `ParsekFlight` resolves and applies active/background caches during tree finalization while preserving fresh-live precedence.

The historical reliability gaps now resolve as follows:

End mode	Landed behavior	Reliability contract
Scene exit, live vessel exists	Fresh scene-exit finalizer appends predicted/extrapolated segments and terminal state.	Fresh live finalization wins over any cache.
Scene exit, vessel missing	Tree finalization resolves the active/background cache and applies it before inference.	Missing runtime objects no longer imply weak last-sample finalization when a cache exists.
Active crash before scene exit	`OnVesselWillDestroy` forces a cache refresh; post-destruction finalization consumes the cache after crash coalescing.	Destroyed active recordings keep a cache-extended endpoint instead of relying only on the final sampled point.
Background vessel destroyed	`OnBackgroundVesselWillDestroy` refreshes/flushes and keeps the cache; `ParsekFlight.DeferredDestructionCheck` applies it only after true destruction is confirmed, then persists the sidecar.	Background destroyed recordings get durable terminal state and synthetic tail metadata without treating false unload/docking signals as terminal.
Debris TTL or out-of-bubble end	Background TTL/missing-vessel paths refresh/apply cache before removing the recording from `BackgroundMap`.	KSP deletion endpoints are preserved and capped to stock deletion UT.
Focus switch / backgrounding	Focus switch transfers cache ownership from `FlightRecorder` to `BackgroundRecorder`; it does not finalize by itself.	A later unload/destroy path still has the last focused projected ending.
UI maneuver node present	`PatchedConicSnapshot` reports `EncounteredManeuverNode`; the finalizer discards stock node-contaminated tails and falls back to current-state propagation.	Planned UI burns remain hypothetical and cannot truncate or redirect finalization.

Terminology

- **Finalization** - sealing a `Recording` with a terminal state, authoritative `ExplicitEndUT`, terminal orbit/surface metadata, and any synthetic predicted tail needed for playback.
- **Synthetic tail** - predicted or extrapolated trajectory data after the last real recorded sample. It is playback/terminal metadata, not a replacement for actual history.
- **Finalization cache** - an in-memory, last-known-good synthetic tail for one live recording owner. It is refreshed while the vessel exists and consumed when the vessel ends unexpectedly.
- **Active recorder cache** - cache owned by the focused `FlightRecorder`.
- **Background cache** - cache owned by `BackgroundRecorder` per tracked background PID/recording id.
- **Premature end** - any recording-end path where Parsek cannot rely on a fresh live-vessel scene-exit finalizer: crash, KSP delete, physics-bubble unload, debris TTL expiry, focus-switch abandonment, or teardown with unavailable runtime APIs.
- **Actual projected chain** - the future path implied by the vessel's current state vector and current thrust/coast state. It explicitly excludes maneuver nodes and other UI planning tools.

Mental Model

Recording has two independent products:

```
actual history, sampled as the player flies
  -> trajectory points, orbit checkpoints, part events

latest projected ending, refreshed while the vessel exists
  -> predicted/extrapolated tail, terminal UT, terminal state
```

Only the first product describes what the vessel already did. The second product answers: "If Parsek loses this vessel now, what would KSP consider the natural end from this point?"

The cache is a safety net:

```
while recording:
    sample real history normally
    every 5 seconds, or on a meaningful state event:
        refresh the finalization cache from the vessel's current state
        (live or on-rails)

when a recording ends:
    if live vessel is still valid:
        compute one fresh finalization result and apply it
    else if a cache exists:
        trim the cached tail to the recording's last real sample UT and apply it
    else:
        use today's inference fallback and log the degraded result
```

KSP's unload model gives the policy its shape. Unloaded vessels move on rails; atmospheric forces are not applied to distant vessels. Community-facing KSP API docs describe unloaded/on-rails motion as orbital-only rather than full-atmosphere physics, and KSP players document the stock auto-delete behavior for objects below the body-specific deletion altitude and outside the active vessel's physics bubble. Parsek mirrors that gameplay outcome: in-atmosphere unfocused deletion is `Destroyed`; stable vacuum orbit is `Orbiting`.

Gameplay Scenarios

Booster Separation in Atmosphere

The player launches a two-stage rocket. The first stage has a probe core and parachutes, so it becomes a controllable sibling at staging. The player continues flying the upper stage. The booster falls behind, leaves the loaded physics bubble, and stock KSP deletes it once it is outside range and still below the atmospheric deletion threshold.

Expected Parsek behavior:

- The booster background recording has its own finalization cache.
- While the booster is loaded, the cache refreshes from its falling state.
- When KSP unloads/deletes the booster, Parsek consumes the cache, appends the predicted/descent tail, sets `TerminalStateValue = Destroyed`, and records the terminal UT.
- The booster appears as an Unfinished Flight if its parent branch has a Rewind Point.

Mun Lander and Orbiter

The player undocks a Mun lander from an orbiter. The lander is active; the orbiter is a background recording in a stable Mun orbit. The player lands and exits to the Space Center.

Expected Parsek behavior:

- The lander finalizes from a fresh live vessel on scene exit.
- The orbiter background cache says the orbiter remains in stable orbit.
- If the orbiter is still live, finalization can refresh it; if not, the cache seals it as `Orbiting`.
- No synthetic atmospheric destruction is invented for a stable vacuum orbit.

Scene Exit Mid-Burn

The player clicks Space Center during ascent while engines are firing.

Expected Parsek behavior:

- The cache may be up to 5 seconds stale, but the scene-exit path makes one fresh live-vessel finalization attempt before teardown.
- If the live attempt succeeds, it wins over the cache.
- If KSP has already torn down the vessel/runtime, the cache is good enough to avoid a frozen mid-burn endpoint.

Focused Crash

A re-entry craft hits terrain while focused. `onVesselWillDestroy` fires, the part-death cascade begins, and a post-destruction tree merge may happen before scene exit.

Expected Parsek behavior:

- The last pre-impact cache already predicts destruction near the crash UT.
- The crash/coalescer finish path consumes that cache instead of relying only on the final sampled point.
- The recording gets `Destroyed`, a terminal UT, and a short tail that carries playback into the impact/destruction endpoint.

Vessel Switch Abandonment

The player switches away from a controllable sibling and never returns to it. It backgrounds, goes on rails, then later unloads or is destroyed by stock rules.

Expected Parsek behavior:

- Switching focus alone is not a terminal event.
- The recording owner changes from active to background, but the cache follows the recording id.
- When the background vessel actually ends or becomes unreachable, Parsek consumes that recording's cache.

Rewind to Separation Dependency

A staged booster ended in the background and the player wants to re-fly it. The UI must know whether the booster is genuinely crashed/destroyed, still orbiting, safely landed, or unfinalized.

Expected Parsek behavior:

- `TerminalKindClassifier` receives a reliable terminal state instead of stale last-sample inference.
- Unfinished Flights membership is based on a committed finalization contract, not on scene-exit timing.

Data Model

The cache is in-memory only. It is not serialized, and it does not change the recording sidecar format. Recordings are still committed only when a recording ends.

Landed shape:

```
internal sealed class RecordingFinalizationCache
{
    public string RecordingId;
    public uint VesselPersistentId;
    public FinalizationCacheOwner Owner;      // ActiveRecorder, BackgroundLoaded, BackgroundOnRails
    public FinalizationCacheStatus Status;    // Empty, Fresh, Stale, Failed

    public double CachedAtUT;
    public float CachedAtRealtime;
    public string RefreshReason;
    public string DeclineReason;

    public double LastObservedUT;
    public string LastObservedBodyName;
    public Vessel.Situations LastSituation;
```

```
    public bool LastWasInAtmosphere;
    public bool LastHadMeaningfulThrust;
    public string LastObservedOrbitDigest;

    public double TailStartsAtUT;
    public double TerminalUT;
    public TerminalState? TerminalState;
    public string TerminalBodyName;
    public RecordingFinalizationTerminalOrbit? TerminalOrbit;
    public SurfacePosition? TerminalPosition;
    public double? TerrainHeightAtEnd;

    public List<OrbitSegment> PredictedSegments; // isPredicted = true
}
```

Ownership:

- `FlightRecorder` owns one cache for the focused recording it is currently sampling.
- `BackgroundRecorder` owns a cache per `BackgroundMap` entry or per loaded/on-rails state. The cache is keyed by recording id and vessel PID so active → background transfer preserves the last focused cache; background → active promotion starts a fresh active cache from the live vessel.
- `VesselPersistentId = 0` means unknown legacy/test identity. Cache application still requires `RecordingId` to match, and when both sides have a nonzero vessel PID, those PIDs must match too.
- The cache payload is copied into the target `Recording` only when a consumer finalizes that recording.

No `Recording` field is needed for the cache itself. Applying a cache reuses existing persisted fields: `OrbitSegments`, `ExplicitEndUT`, `TerminalStateValue`, terminal orbit fields, `TerminalPosition`, and `TerrainHeightAtEnd`.

Behavior

Refresh Policy

Refresh every 5 seconds while a recording has a live vessel reference, plus forced refreshes on meaningful state changes:

- recording start or promotion from background
- active → background transition
- throttle crossing idle/active thresholds
- engine or RCS activity while loaded
- atmosphere entry or exit
- SOI change
- go-on-rails or go-off-rails
- vessel situation change
- staging, undock, EVA, joint break, and part death
- time-warp rails/checkpoint boundaries
- scene-exit/background-commit finalization entry, where fresh live finalization or background force-refresh runs before any fallback consumes stale data

Refresh early-outs when the vessel state digest has not meaningfully changed. A stable coasting vacuum orbit does not need a new cache every 5 seconds if the orbit, body, situation, throttle state, and terminal result are unchanged.

Actual Chain Only

Finalization must ignore KSP maneuver nodes. A maneuver node is an editor/planning UI object, not recorded trajectory data and not something the vessel will actually do unless the player later burns.

Landed behavior detects maneuver-node boundaries in `PatchedConicSnapshot` with `EncounteredManeuverNode`. When a stock patched-conic tail is node-contaminated, `IncompleteBallisticSceneExitFinalizer` discards that tail and falls back to current-state propagation through Parsek's conic/ballistic path. The old `StoppedBeforeManeuver` terminal behavior is retired because it turned a hypothetical UI node into a real terminal boundary.

Active Recorder Refresh

The active recorder attempts the highest-fidelity cache:

- For live focused vessels, refresh from the live vessel state and stock node-free conic data when available.
- During powered flight or atmospheric flight, the cache is expected to change; refresh by cadence and forced triggers.
- During vacuum coast with no meaningful thrust, the cache stabilizes; repeated refreshes are skipped unless an event invalidates the digest.

Background Recorder Refresh

Background recorders do not have the same solver access as the active vessel. They still cache a reliable terminal outcome:

- Loaded background vessels in the physics bubble can refresh from live vessel/orbit state.
- On-rails background vessels can refresh from `vessel.orbit` and existing on-rails checkpoints.
- Stable vacuum orbits cache `Orbiting` plus terminal orbit metadata. "Stable" means a bound orbit whose periapsis clears the body's periapsis floor (`OrbitClearance` : the atmosphere top, or the highest terrain on an airless body), the same line the terminal-orbit spawn check reads (operator ruling 2026-09-27, flight-recorder design 13.5). An extrapolated `Orbiting` whose last segment's periapsis is under the floor (the extrapolator ran out its horizon on an airless-body orbit whose sampled ground track missed the peaks) is downgraded to `SubOrbital` .
- Atmospheric or suborbital cases cache `Destroyed` at the predicted KSP deletion/destruction endpoint. If KSP deletes the vessel before the extrapolator's endpoint, the deletion event wins the terminal state but the cached tail still explains the end.

This does not create a cross-scene background recorder. The cache only protects the in-flight session that is already recording.

Applying a Cache

When a cache is consumed:

1. Find the last real authored UT on the `Recording` from `Points` , `TrackSections` , and non-predicted `OrbitSegments` .
2. Drop cached segments that end before that UT.
3. Clamp the first retained segment start to the last real UT when safe.
4. Append retained predicted segments with `isPredicted = true` .
5. Set `ExplicitEndUT` to the cache terminal UT if it extends the recording.
6. Set `TerminalStateValue` , terminal orbit/surface metadata, and terrain height when present.
7. Mark files dirty if the recording has already flushed sidecar data.
8. Emit a single summary log line naming the cache age, freshness, source, and consumer path.

Fresh live finalization has precedence over cache application. Cache application has precedence over trajectory-only inference. Trajectory-only inference remains the last-resort fallback.

Consumer Paths

The implementation consumes cache at these seams:

- scene exit for active and background recordings when the live finalizer declines or the vessel is missing
- manual commit or revert commit when a leaf/background vessel is already missing but has a usable cache
- active vessel destruction after crash coalescing decides the recording/tree should finalize
- confirmed background destruction through `ParsekFlight.DeferredDestructionCheck` after `OnBackgroundVesselWillDestroy` has refreshed/flushed and kept the cache
- debris TTL, out-of-bubble, and missing-vessel termination in `BackgroundRecorder.CheckDebrisTTL`
- vessel-unloaded paths that remove or strand a background recording
- any stop/abandon path that removes a recording from `BackgroundMap` or ends a live recorder without a fresh vessel

Focus switching is not itself a consumer unless the switch path actually ends the old recording. In the normal tree model, switching transfers cache ownership to the background recorder instead.

Edge Cases

1. **Live scene exit with active vessel present.** Fresh finalization runs and wins. Cache is only a fallback.

2. **Scene exit after vessel object vanished.** Cache applies. If no cache exists, trajectory inference applies and logs degraded finalization.
3. `FlightGlobals` **unavailable in teardown/headless seam.** Cache applies without touching runtime-only APIs. If the cache is empty, existing guarded decline remains.
4. **Manual commit after a sibling already unloaded.** Cache applies before the non-scene-exit missing-vessel path stamps a weak destroyed fallback.
5. **Player has a planned maneuver node.** The node is ignored. The cached chain is the real coast/powered trajectory from current state, not the UI plan.
6. **Burn in progress.** Cache refreshes by cadence and throttle/engine triggers. Scene exit attempts one fresh refresh before consuming anything.
7. **Thrust stops in vacuum.** Forced refresh captures the new coast. Later refreshes can early-out until SOI/warp/orbit changes.
8. **Atmospheric unpowered flight.** Cache refreshes while loaded; if KSP unloads/deletes the vessel, terminal state is `Destroyed`.
9. **Stable vacuum orbit.** Cache can settle as `Orbiting`; no synthetic destruction or 50-year horizon spawn is invented for ordinary stable orbits.
10. **Suborbital vacuum arc with periapsis in atmosphere.** Cache predicts `Destroyed` even if the vessel is currently above atmosphere.
11. **Non-atmospheric body impact.** Cache predicts ground impact using the same ballistic terrain/sea-level fallback policy as the existing extrapolator.
12. **Background on-rails vessel lacks patched-conic solver.** Use orbit-state and ballistic/conic extrapolation only.
13. **SOI transition between refreshes.** Last cache may be stale; SOI event forces a refresh. If the vessel disappears in the same frame, the previous cache is still better than no tail and logs as stale.
14. **Crash coalescer still pending.** Do not apply cache until the coalescer has decided which tree leaves are terminal. The cache supplies the endpoint, not the structural BranchPoint decision.
15. **Debris TTL expiry while vessel still exists.** Consume cache if it is fresher than the TTL cutoff; otherwise refresh once from the live debris vessel before ending the recording.
16. **KSP deletes an unfocused atmospheric vessel earlier than Parsek's ballistic impact.** Stock outcome wins: terminal state `Destroyed`. Terminal UT is the earlier of deletion UT and predicted impact UT, and the cached tail is trimmed to that terminal UT. If no predicted impact UT exists, deletion UT is the terminal UT.
17. **Cache older than allowed freshness window.** Apply only with a `Stale` log if no live refresh is possible; otherwise refresh first.
18. **Recording has no real samples.** Do not append a free-floating tail. Mark degraded/destroyed according to the existing zero-point/debris policy.
19. **Cache terminal UT is before the last real sample.** Reject the cache and fall back to live/inference. This catches stale or mismatched ownership.
20. **Recording is already finalized.** Do not overwrite terminal metadata unless the consumer is explicitly repairing a known degraded finalization and logs the replacement.
21. **Hard quit without commit.** Cache is in-memory, so it is lost. This is acceptable: Parsek commits on recording end, not continuously through power loss.
22. **Old committed recordings.** No migration. They keep whatever terminal data they already have.
23. **Many background vessels.** Refresh is per recording, but digest early-outs and 5-second cadence keep cost bounded. Implementation logs aggregate refresh counts rather than per-vessel spam.
24. **Non-final chain segment in a live tree.** A Re-Fly or resumed tree spliced from the committed tree can carry an optimizer chain HEAD (no terminal, no child branch point) next to its successor. It is never finalized as a leaf (`ParsekFlight.IsNonFinalChainSegmentInTree`): its terminal lives on the final segment. Exempt: the tree's active recording, and a segment this session recorded past its successor's start.
25. **Live vessel owned by later committed history.** A pid match does not date the live vessel. When it is the terminal spawn of a different committed recording that ends after this one (`TreeCommitService.FindLaterCommittedSpawnOwner`, identity through `VesselLaunchIdentity.LiveVesselIsRecordedSpawn`: a unique spawn pid matches alone, an adoption stamp also needs the launch guid not to conclusively differ, an unknown guid falls back to pid-only), the recording is not finalized from it and keeps its committed terminal, snapshot and orbit. A chain-tip spawn keeps both the craft-baked pid and the launch guid, so a launch-guid check alone cannot tell this case apart. The tree's active recording is exempt.

What Doesn't Change

- Actual sampled history is still authoritative. Synthetic tails never rewrite existing trajectory points.
- Recording sidecar format does not change for the cache. It is in-memory only.
- `OrbitSegment.isPredicted` remains the persisted marker for predicted/extrapolated segments that are applied to a committed recording.

- Committed recordings remain immutable after commit. Cache application happens before commit or as part of the existing finalization path.
- Background recording remains an in-flight, physics-bubble system. This design does not add cross-scene background simulation.
- Parsek does not simulate full atmospheric drag for unloaded vessels. It mirrors stock KSP's practical outcome for unfocused atmospheric deletion/destruction.
- Rewind to Separation UI and supersede semantics do not change here. They consume better terminal data from the finalization cache without changing their UI contract.
- ERS/ELS access rules do not change. The cache is pre-commit finalization state and must not introduce new raw `RecordingStore.CommittedRecordings` or `Ledger.Actions` readers outside the existing grep-audit allowlist.

Backward Compatibility

There is no save migration and no new serialized cache node. Existing recordings keep their terminal state and predicted tail data as written.

If later work discovers it needs a small persisted marker for "finalization was degraded", that must be a separate design update. The locked decision for this feature is in-memory cache only.

Diagnostic Logging

Use subsystem tags that make the finalization decision reconstructable from `KSP.log`:

- `[FinalizerCache]` for refresh, cache status, cache application, cache rejection, and ownership transfer.
- `[Extrapolator]` for ballistic/conic propagation, terminal reason, limits, and failures.
- `[PatchedSnapshot]` for node-free stock chain capture and maneuver-node exclusion decisions.
- `[BgRecorder]` for background cache application and persistence paths, including confirmed destruction, unload, TTL, and out-of-bubble.
- `[Flight]` for tree-level finalization decisions and crash-coalescer timing.

Required log events:

- cache refresh attempt: recording id, vessel pid, owner, UT, reason, digest changed yes/no
- refresh decline: reason, whether existing cache remains usable
- cache stored: terminal state, terminal UT, segment count, source path, elapsed milliseconds
- ownership transfer: active → background or background → active with recording id and pid
- consumer entry: consumer path, live vessel found yes/no, cache status, cache age
- cache application summary: appended segment count, old/new `ExplicitEndUT`, terminal state
- cache rejection: stale, mismatched recording id/pid, terminal before last sample, empty payload, or already finalized
- degraded fallback: no live vessel and no usable cache, inference path used
- maneuver-node exclusion: node count or transition detected, selected node-free strategy, restore success/failure

Per-frame refresh checks must use rate-limited logs or aggregate summaries.

Coverage Contract

The feature landed with headless unit/integration/log-assertion coverage plus in-game runtime canaries. Future behavior changes should keep that coverage shape and follow the project testing rule.

Headless Coverage Behaviors

The unit and integration suites cover:

- cached tails append after the last real sample without overwriting or duplicating authored trajectory history
- stale caches cannot shorten a recording by moving the terminal UT before the last real sample
- active → background transfer preserves the old focused recording cache
- stale caches apply only when the consumer explicitly allows stale fallback and no live refresh is possible
- stock atmospheric deletion forces `Destroyed` rather than `Orbiting`

- stable coast avoids unnecessary refresh churn
- planned UI maneuver nodes cannot truncate finalization or become terminal data
- stable background/on-rails orbiters classify as `Orbiting`
- falling atmospheric/suborbital background vessels classify as destroyed or degraded instead of null/orbiting
- cache application marks sidecar data dirty/persistent when background finalization has already flushed files

Integration Tests

- Scene-exit finalization falls back to cache when `FlightGlobals` probe declines.
- Manual commit after a background vessel unloads applies cache instead of a generic missing-vessel fallback.
- Active crash path consumes cache after crash coalescer completion.
- Confirmed background destruction applies cache in `DeferredDestructionCheck` after `OnBackgroundVesselWillDestroy` refreshes/flushes and keeps the cache.
- Debris TTL out-of-bubble path consumes cache before removing `BackgroundMap`.
- Focus switch active → background → unload keeps one recording id and one cache.

Log Assertion Tests

- Refresh/store summary includes owner, recording id, terminal state, terminal UT, and reason.
- Cache rejection logs the exact rejection reason.
- Degraded fallback logs when no live vessel and no cache exist.
- Maneuver-node exclusion logs the node-free strategy used.
- Background consume paths log aggregate counts instead of one noisy line per frame.

Runtime Canaries And Manual Checks

The `RecordingFinalization` in-game category currently has synthetic FLIGHT canaries for:

- destroyed background cache trimming to stock deletion UT
- stable background cache applying `Orbiting` to a missing vessel
- non-scene active crash finalization consuming active cache before the destroyed fallback

The gameplay checklist in `dev/manual-testing/test-recording-finalization-cache.md` covers atmospheric booster unload/delete, stable background orbiters, scene exit mid-burn, planned maneuver-node ignoring, focused crash, log sweeps, and cadence calibration.

Code Layout

The completed phase plan is archived in `dev/done/plans/recording-finalization-reliability.md`. The landed code is split across these responsibilities:

1. `RecordingFinalizationCache` - in-memory identity, freshness, terminal, and predicted-tail payload.
2. `RecordingFinalizationCacheProducer` - active/background refresh cadence, stable-orbit cache production, atmospheric deletion cache production, and digest/no-op handling.
3. `RecordingFinalizationCacheApplier` - pure apply/reject logic, predicted-tail trimming, terminal metadata stamping, and dirty-sidecar marking.
4. `FlightRecorder` and `BackgroundRecorder` - cache ownership, refresh producers, active/background transfer, and background end consumers.
5. `ParsekFlight` and `IncompleteBallisticSceneExitFinalizer` - fresh-live precedence, scene-exit fallback consumption, crash/deferred-destruction consumption, and maneuver-node-safe finalization.

References

- `dev/done/plans/incomplete-ballistic-extrapolation.md` - shipped scene-exit predicted-tail plan and current extrapolator context.
- `dev/done/plans/recording-finalization-reliability.md` - completed implementation phase plan.
- `dev/manual-testing/test-recording-finalization-cache.md` - runtime/manual validation checklist.
- `dev/development-workflow.md` - design and plan/build/review workflow.

- kOS Vessel Load Distance docs - describes loaded/unloaded and packed/on-rails behavior in KSP.
- KSP forum discussion: 25km auto delete - community documentation of stock atmospheric auto-delete rules.
- KSP forum discussion: unloaded ships do not aerobrake - community discussion of unloaded vessels not receiving normal aerobraking.
- KSP Wiki: Atmosphere - body atmosphere overview and drag context.